

Elemental Quest — Phase 2 Implementation Report

CMPT 276 — Introduction to Software Engineering | Group 12
Team Members: Bhav Wadhwa, Kannan Jayakumar, Kanika Soni, Xavier

1. Overall Approach to Implementing the Game

1.1 Game Overview

Crafted as a 2D tile-based puzzle-action title, Elemental Quest utilizes the foundational Java standard library to create a highly responsive and synchronized gameplay experience. Players navigate two elemental heroes, Ember and Aqua, across grid-centric environments. Passing through each level depends on the strategic collection of crystals, neutralizing environmental hazards, and overcoming enemies to reach locked exit gates before time runs out. To provide a balanced progression, the game features nine unique level designs distributed across three distinct modes: individual play as Ember, individual play as Aqua, and a two-player cooperative experience.

1.2 Architecture: Bottom-Up, Layered Design

The implementation followed a bottom-up, layered architecture organized into nine clearly separated packages. This structure enabled parallel development with minimal interference. The project contains 32 Java source files totalling approximately 4,454 lines of code.

Package	Responsibility
game.core	Entry point (GameManager), game-state machine, timer, scoring, audio
game.character	Player characters (Ember, Aqua) via abstract base class
game.collectible	Elemental crystals (required) and bonus crystals (score only)
game.combat	Projectile system for ranged attacks (fireballs/waterballs)
game.enemy	Enemy entities (ShadowSentinel) and the Hazard interface
game.gate	Exit gates (FireGate, WaterGate) that unlock when crystals collected
game.level	Board 2D grid, Cell tiles, and Level configurations
game.ui	All rendering, menus, and user interaction via Java Swing
game.util	Shared enumerations (Direction, ElementType, GemType)

1.3 Construction Milestones

The development process was structured into three primary stages across two major milestones:

Milestone 1 (Midway): Focused on finishing Stages 1 and 2. This included establishing all fundamental engine classes, game logic, entity behaviors, and combat mechanics. By this point, the game was entirely functional in a console environment with all rules active, meeting the target schedule.

Milestone 2 (Final): Involved completing Stage 3, which centered on the graphical user interface, audio systems, and final refinements. The project concluded with a fully playable, polished, and deliverable product.

Detailed Breakdown of Development Stages:

Stage 1 — Foundation: Created the core architecture, including the Level, Board, Cell, and Entity classes, as well as the GameManager and ElementalCharacter base.

Stage 2 — Game Logic: Built the interactive systems, such as enemy AI, collectible mechanics, and ranged combat, all integrated across nine unique level layouts.

Stage 3 — UI & Polish: Constructed the graphical interface using a Swing-based GamePanel optimized for 60 FPS, incorporating the PixelArtRenderer and the SoundManager for audio control.

1.4 Key Gameplay Systems

Input Orchestration and Simultaneous Control: To create a simultaneous co-operative play, the game implements a dual-input configuration: Player 1 utilizes the WASD keys with Space for abilities and Q for ranged attacks, while Player 2 uses the arrow keys, Enter, and Shift. This orchestration is handled by a *heldKeys Set<Integer>* mechanism, which prevents hardware rollover issues and ensures responsive, concurrent interactions for both players.

Resource-Limited Abilities and Projectile Combat: Each character is provided three ability charges per level, enabling strategic terrain modification to navigate hazards. Complementing this is a ranged projectile system allowing players to neutralize enemies and obstacles, governed by a five-second cooldown to maintain gameplay balance.

Victory Conditions and Scoring Penalties: Success requires the collection of all essential elemental crystals and reaching the designated exit gates within the timer; notably, co-operative sessions mandate simultaneous gate occupancy. To enforce precision, a 100-point penalty is assessed for collecting incompatible elemental gems, with a negative total score resulting in an immediate game-over state.

2. Adjustments and Modifications to the Initial Design

2.1 Structural Changes

- **Enemy Interface Unused:** ShadowSentinel implements Entity and Hazard directly instead of the planned Enemy interface, since movement logic is tightly coupled to Board state.
- **Player as Composition Wrapper:** Player composes Ember and Aqua and delegates input, rather than inheriting from Entity — necessary for coordinating two independent grid entities in co-op.
- **Simplified Enums:** Scoped from 4 elements to 2 for higher quality within available time. Unused values remain for future expansion.

2.2 Gameplay Changes

- **Projectile Combat Added:** Ranged combat via Projectile class with 5-second cooldown.
- **Ability Charge Limit (3/level):** Creates resource-management decisions.
- **Score Penalty in Co-op:** -100 for collecting opposing player's crystal type.
- **Enemy Respawn:** ~5 seconds after death at random safe location ≥ 5 tiles from any player.

2.3 UI Changes

Cyberpunk neon aesthetic(dark purple/blue palette with neon accents). Resizable game window via AffineTransform dynamic scaling. Tileset fallback system ensures full procedural rendering if sprite sheets are missing.

3. Management Process and Division of Roles

3.1 Team Member Contributions

Team Member	Primary Responsibilities
Bhav Wadhwa	Core game classes (GameManager, GameState, Player, Ember, Aqua); gate system; audio (SoundManager); bug fixing; level design; merge conflict resolution
Kannan Jayakumar	UI module (initial impl.); UI documentation; gameplay logic fixes; special gem counter; UI mockup design
Kanika Soni	Javadoc documentation; hazard and level implementation; main menu styling; border animation; window resizing; use case specs
Xavier	Maven build system; package hierarchy unification; UML diagram; cross-environment integration testing

3.2 Development Workflow

Version Control: The team used a centralized Git workflow on the SFU GitHub instance with main as the integration branch. Team members worked on features locally and pushed to main, resolving merge conflicts as they arose. GitHub Issues were used to track bugs and feature requests (6 issues filed: 4 closed, 2 open at time of submission). The commit history reflects incremental integration, with dedicated “FinalVersion” commits marking stable milestones.

Communication & Coordination: The team coordinated via a shared Discord server with dedicated channels for bug reports, feature discussion, and merge announcements. Brief check-in meetings were held before major integration points to align on priorities. This was especially critical when Xavier applied the Maven restructure over existing code, which required a dedicated team session to resolve merge conflicts across every source file.

Phase Progression: Development followed the 3-stage plan aligned with the two milestones. Stages 1 and 2 were completed by the midway-progress deadline, ensuring all core logic was operational. Stage 3 (UI, rendering, audio, polish) was completed in the final push, followed by a focused bug-fixing period addressing issues such as fireball speed, spike interactions, font consistency, HUD alignment, and dynamic scaling.

4. External Libraries

Library	Ver.	Purpose	Justification
Java Swing	JDK 17	GUI framework, rendering, event handling	Swing is part of the standard Java library and requires no additional dependencies. It provides sufficient capability for 2D tile-based rendering via Graphics2D, supports CardLayout for multi-screen navigation, and javax.swing.Timer for the game loop. Chosen over JavaFX for simpler setup, lighter footprint, and team familiarity.
mp3spi	1.9.5.4	MP3 audio playback via Java Sound SPI	Java's javax.sound.sampled API does not natively support MP3. The mp3spi library adds an MP3 decoder as a Service Provider Interface plugin, enabling seamless playback through the standard AudioSystem API. Preferred over JLayer for transparent integration.
Maven	3.x	Build system and dependency management	Maven manages the mp3spi dependency declaratively, standardizes the build process, and produces a self-contained fat JAR via maven-shade-plugin. The shade plugin also merges SPI service files — critical for mp3spi auto-discovery. Ensures all team members can build with a single mvn package command.

Note: All game rendering — pixel-art tiles, animations, glow effects, HUD — is produced entirely using Java's built-in Graphics2D API. No external graphics libraries were used.

5. Strategies for Improving Code Excellence

5.1 Standards for Documentation

To maintain transparency, every class features extensive Javadoc comments, including headers that outline design principles, intended purposes, and core behaviors. Detailed documentation using @param and @return tags is provided for all critical methods. Furthermore, internal code comments clarify complex systems, such as collision detection sequences, ability prioritization, and safe-zone calculations for respawning. For improved navigability, substantial source files including the 1,000+ line PixelArtRenderer are organized into distinct modules using Unicode headers.

5.2 Architecture and Design Patterns

Pattern	Implementation
Template Method	ElementalCharacter defines shared logic; Ember/Aqua only override useAbility()
Singleton	SoundManager uses lazy init for global audio access
Composition over Inheritance	Player composes Ember and Aqua rather than inheriting from Entity
Interface-Driven Design	Entity, Collectible, Hazard, Gate interfaces ensure loose coupling
Static Nested Classes	ElementalCrystal/BonusCrystal and FireGate/WaterGate keep related types together

5.3 Defensive Programming

Bounds checking via `Board.isValidPosition()` before every grid access. Null safety checks throughout movement and interaction code. `ConcurrentModificationException` prevention via snapshot-copied lists (`new ArrayList<>(list)`) during iteration. Graceful audio fallback (SoundManager catches all exceptions, game runs silently if audio is missing). Tileset fallback to complete procedural rendering if PNGs are missing.

5.4 Gameplay Robustness

Movement debouncing via `heldKeys` set with 120ms `MOVE_DELAY_MS` timestamps. 1-second damage cooldown prevents instant death on overlapping hazards. Mutually exclusive terrain (setting lava clears water and vice versa). Snapshot-based iteration on all per-tick update loops.

5.5 Build System Quality

`pom.xml` enforces Java 17 as source and target. `maven-shade-plugin` configured with `ServicesResourceTransformer` to correctly merge SPI service descriptors for `mp3spi` auto-discovery.

6. Challenges

Challenge 1: Two-Player Mode on a Single Keyboard

Implementing simultaneous two-player controls on a shared keyboard proved to be one of the most technically demanding aspects. Hardware keyboard rollover limits (3–6 simultaneous keys), independent movement timing, and the synchronized win condition (both players must stand on their gates simultaneously) all required careful engineering. Solution: A `heldKeys Set<Integer>` tracks all depressed keys, eliminating OS key-repeat ambiguity. Per-character `lastMoveTime` timestamps govern independent movement cadence. The win condition evaluates both gate states every frame.

Challenge 2: Merge Conflicts and Mid-Project Restructuring

With four developers committing to main on interconnected files, merge conflicts were frequent. The most disruptive episode was Xavier restructuring the entire codebase into a Maven-based multi-package hierarchy mid-development, requiring every file to be moved, all imports updated, and accidentally dropped Javadoc/logic recovered. Solution: The team held dedicated merge sessions, testing each component in isolation before integrating back to main. This was a valuable lesson in agreeing on project structure before implementation begins.

Challenge 3: Game Loop and Multi-Timer Coordination

The game uses multiple concurrent `javax.swing.Timer` instances (16ms game loop, 50ms animation, 320ms enemy movement, 5s projectile cooldown, ~5s enemy respawn). Without care, an enemy could take a movement step between a projectile's position update and its collision check, causing "ghost hits." Solution: Snapshot-based iteration for all entity lists and careful sequencing of update operations within the 16ms tick eliminated race conditions. Cooldown guards prevented "double damage" from overlapping hazards.

Challenge 4: Procedural Rendering Without External Graphics Libraries

Building a full cyberpunk aesthetic (auras, trails, HUD, animated scan lines) using only `Graphics2D` was ambitious. `PixelArtRenderer.java` alone handles all tile types, both characters, enemies, crystals, gates, projectiles, and the full HUD. Early versions suffered frame-rate drops from per-frame `Font` and `AlphaComposite` allocation. Solution: Caching `Font`, `BasicStroke`, and `AlphaComposite` instances as class-level constants was

essential for maintaining smooth 60 FPS. Anti-aliasing was disabled for tile interiors (preserving the pixelated look) but kept enabled for text.

Challenge 5: Level Design and Balancing

Hardcoding 9 level configurations while ensuring each was fun, fair, and appropriately challenging took more time than anticipated. Early versions were either trivially easy or frustratingly unfair. Solution: The team conducted multiple rounds of structured playtesting, each member played every level and logged pain points. Iterating on enemy placement, crystal counts, time limits, and board dimensions until each level felt balanced. In hindsight, even a simple level editor would have significantly accelerated this process.